

GATE DATA SCIENCE & AI · EXAM PREP

GATE DA Study Guide

Nested Loops Complexity · Master Theorem (5 Recurrences) ·
Uninformed Search — UCS

- ▶ Nested Loops — Multiply, Additive, Log, Harmonic & Geometric Patterns
- ▶ Nested Loops — 10 PYQs + 10 New Practice Questions
- ▶ Master Theorem — Statement, 3 Cases & When It Fails
- ▶ Master Theorem — 5 Solved Examples + 10 PYQs + 10 Practice Questions
- ▶ Uninformed Search: UCS — Mechanics & Dijkstra Equivalence
- ▶ UCS — 10 PYQs + 10 New Practice Questions

PART 1 — Nested Loops Complexity

1.1 Core rules (recap)

- **Independent nested loops** (inner bound doesn't depend on outer variable) → **multiply** the individual loop complexities.
- **Dependent, additive bound** (e.g., `j` from `1` to `i`) → usually still $\Theta(n^2)$ via arithmetic series ($\text{sum} = n(n+1)/2$).
- **Multiplicative/divisive counter** (`i *= 2`, `i /= 2`) → contributes a **log n** factor instead of n .
- **Sequential (non-nested) blocks** → **add** complexities, keep only the dominant term.
- **Harmonic-style bound** (`j` from `1` to `n/i`) → $\text{sum} = n \cdot (1 + 1/2 + 1/3 + \dots + 1/n) = n \cdot H_n = \Theta(n \log n)$.
- **Shrinking geometric bound** (counter halves each outer step, inner loop runs full current value) → total work is a geometric series that **collapses to $\Theta(n)$** , not $\Theta(n \log n)$ — a classic trap.

1.2 10 GATE-style PYQs

Q1 `for(i=1;i<=n;i++) for(j=1;j<=n;j++) sum++;` — Complexity?

(a) $\Theta(n)$

(b) $\Theta(n^2)$ ✓

(c) $\Theta(n \log n)$

(d) $\Theta(2^n)$

ANSWER (b) — two fully independent loops of size n , multiply: $n \times n = \Theta(n^2)$.

Q2 `for(i=1;i<=n;i++) for(j=i;j<=n;j++) sum++;` — Complexity?

(a) $\Theta(n)$

(b) $\Theta(n \log n)$

(c) $\Theta(n^2)$ ✓

(d) $\Theta(n^3)$

ANSWER (c) — triangular iteration; total iterations = $n + (n-1) + \dots + 1 = n(n+1)/2 = \Theta(n^2)$.

Q3 `for(i=1;i<=n;i++) for(j=1;j<=n;j++) for(k=1;k<=n;k++) sum++;` — Complexity?

(a) $\Theta(n^2)$

(b) $\Theta(n^3)$ ✓

(c) $\Theta(n^2 \log n)$

(d) $\Theta(2^n)$

ANSWER (b) — three independent loops of size n → n^3 .

Q4 `for(i=1;i<n;i*=2) for(j=1;j<=n;j++) sum++;` — Complexity?

(a) $\Theta(n)$

(b) $\Theta(\log n)$

(c) $\Theta(n \log n)$ ✓

(d) $\Theta(n^2)$

ANSWER (c) — outer runs $\log n$ times (i doubles), inner runs n times each → $n \log n$.

Q5 `for(i=1;i<=n;i++) for(j=1;j<n;j*=2) sum++;` — Complexity?

(a) $\Theta(n)$

(b) $\Theta(n \log n)$ ✓

(c) $\Theta(\log n)$

(d) $\Theta(\log^2 n)$

ANSWER (b) — outer runs n times, inner (doubling) runs $\log n$ times → $n \log n$.

Q6 `for(i=1;i<n;i*=2) for(j=1;j<i;j*=2) sum++;` — Complexity?

(a) $\Theta(\log n)$

(b) $\Theta(n \log n)$

(c) $\Theta(\log^2 n)$ ✓

(d) $\Theta(n)$

ANSWER (c) — outer runs $\log n$ times; for each outer value i (itself a power of 2 up to $\log n$ in magnitude), inner also runs $\sim \log i$ times, summing to $\Theta(\log^2 n)$.

Q7 `for(i=1;i<=n;i++) for(j=1;j<=n/i;j++) sum++;` — Complexity?

(a) $\Theta(n)$

(b) $\Theta(n \log n)$ ✓

(c) $\Theta(n^2)$

(d) $\Theta(\sqrt{n})$

ANSWER (b) — total work = $n/1 + n/2 + n/3 + \dots + n/n = n \cdot H_n = \Theta(n \log n)$ (harmonic series).

Q8 `for(i=n;i>0;i=i/2) for(j=0;j<i;j++) sum++;` — Complexity?

(a) $\Theta(n \log n)$

(b) $\Theta(\log n)$

(c) $\Theta(n)$ ✓

(d) $\Theta(n^2)$

ANSWER (c) — inner loop runs i times for $i = n, n/2, n/4, \dots$; this geometric series sums to $\Theta(n)$, NOT $n \log n$ — classic GATE trap.

Q9 Two separate (non-nested) blocks run one after another: first an $\Theta(n^2)$ loop, then an $\Theta(n \log n)$ loop. Overall complexity?

(a) $\Theta(n^2 + n \log n)$ exactly

(b) $\Theta(n^2)$ ✓

(c) $\Theta(n \log n)$

(d) $\Theta(n^3)$

ANSWER (b) — for sequential blocks you add complexities and keep the dominant term; n^2 dominates $n \log n$.

Q10 `for(i=1;i<=n;i++) for(j=1;j<=2*i;j++) sum++;` — Complexity?

(a) $\Theta(n)$

(b) $\Theta(n \log n)$

(c) $\Theta(n^2)$ ✓

(d) $\Theta(n^3)$

ANSWER (c) — total = $\sum 2i$ for $i=1$ to $n = 2 \cdot (n(n+1)/2) = n^2 + n = \Theta(n^2)$.

1.3 10 New Practice Questions

P1 `for(i=1;i<=n;i++) for(j=1;j<=i*i;j++) sum++;` — Find the complexity.

ANSWER $\Theta(n^3)$ — total = $\sum i^2$ for $i=1..n = n(n+1)(2n+1)/6 = \Theta(n^3)$.

P2 `for(i=1;i<=n;i++) for(j=1;j<=n;j+=2) sum++;` — Find the complexity.

ANSWER $\Theta(n^2)$ — inner loop runs $\sim n/2$ times instead of n ; constant factors don't change the Θ class.

P3 `for(i=n;i>=1;i--) for(j=1;j<=sqrt(i);j++) sum++;` — Find the complexity.

ANSWER $\Theta(n^{1.5})$ — total $\approx \sum \sqrt{i}$ for $i=1..n \approx (2/3)n^{1.5}$.

P4 `for(i=1;i<n;i*=3) for(j=1;j<n;j*=3) sum++;` — Find the complexity.

ANSWER $\Theta(\log^2 n)$ — both loops run $\Theta(\log_3 n) = \Theta(\log n)$ times (base doesn't matter in Θ -notation), multiplied gives $\log^2 n$.

P5 A single loop runs `while(n>0){ n = n - k; iterations++; }` for a fixed constant k . Find the complexity in terms of n .

ANSWER $\Theta(n)$ (more precisely $\Theta(n/k)$, but since k is a constant, this is $\Theta(n)$).

P6 `for(i=1;i<=n;i++){ if(i%2==0) for(j=1;j<=n;j++) sum++; }` — Find the complexity.

ANSWER $\Theta(n^2)$ — the inner loop runs for roughly half the outer iterations ($n/2$ times), each costing n , giving $(n/2) \cdot n = n^2/2 = \Theta(n^2)$.

P7 `for(i=1;i<=n;i++) for(j=1;j<=n;j++) for(k=1;k<=n;k++) for(l=1;l<=n;l++) sum++;` — Find the complexity.

ANSWER $\Theta(n^4)$ — four fully independent nested loops of size n .

P8 `for(i=1;i<=n;i++) for(j=1;j<=log(i);j++) sum++;` — Find the complexity.

ANSWER $\Theta(n \log n)$ — total = $\sum \log(i)$ for $i=1..n = \log(n!) = \Theta(n \log n)$ by Stirling's approximation.

P9 `for(i=1;i*i<=n;i++) for(j=1;j<=n;j++) sum++;` — Find the complexity.

ANSWER $\Theta(n^{1.5})$ — outer loop runs $\sqrt{n}$ times (since $i^2 \leq n$), inner runs n times each $\rightarrow \sqrt{n} \times n = n^{1.5}$.

P10 `for(i=1;i<=n;i++) for(j=i;j<=n;j++) for(k=1;k<=j;k++) sum++;` — Find the complexity.

ANSWER $\Theta(n^3)$ — the two outer loops together give $\Theta(n^2)$ pairs (i,j) with $j \geq i$, and for each the innermost loop costs up to j ($\leq n$); the triple-sum works out to $\Theta(n^3)$.

2.1 Statement (recap)

For $T(n) = a \cdot T(n/b) + f(n)$, $a \geq 1$, $b > 1$, compare $f(n)$ with $n^{\log_b a}$:

Case	Condition	Result
1	$f(n) = O(n^{\log_b a - \epsilon})$	$T(n) = \Theta(n^{\log_b a})$
2	$f(n) = \Theta(n^{\log_b a})$	$T(n) = \Theta(n^{\log_b a} \cdot \log n)$
3	$f(n) = \Omega(n^{\log_b a + \epsilon})$, regularity holds	$T(n) = \Theta(f(n))$

Does NOT Apply When subproblems are of unequal size (e.g., $T(n/3) + T(2n/3)$), the recurrence isn't of divide-and-conquer form (e.g., $T(n-1) + n$), or there's a non-polynomial ("log-factor only") gap between $f(n)$ and $n^{\log_b a}$ — e.g. $T(n) = 2T(n/2) + n \log n$ falls in the gap between Case 2 and Case 3.

2.2 Five Solved Teaching Examples

Solved Example 1 $T(n) = 2T(n/2) + n \rightarrow a=2, b=2 \rightarrow n^{\log_2 2} = n^1$; $f(n) = \Theta(n^1) \rightarrow$ **Case 2** $\rightarrow \Theta(n \log n)$ (merge sort).

Solved Example 2 $T(n) = 4T(n/2) + n \rightarrow a=4, b=2 \rightarrow n^2$; $f(n) = n = O(n^{2-\epsilon}) \rightarrow$ **Case 1** $\rightarrow \Theta(n^2)$.

Solved Example 3 $T(n) = T(n/2) + n \rightarrow a=1, b=2 \rightarrow n^0 = 1$; $f(n) = n = \Omega(n^{0+1})$, regularity holds $\rightarrow$ **Case 3** $\rightarrow \Theta(n)$.

Solved Example 4 $T(n) = 2T(n/2) + n^2 \rightarrow a=2, b=2 \rightarrow n^1$; $f(n) = n^2 = \Omega(n^{1+1})$, regularity: $2(n/2)^2 = n^2/2 \leq c \cdot n^2$ for $c=1/2 < 1$ ✓ $\rightarrow$ **Case 3** $\rightarrow \Theta(n^2)$.

Solved Example 5 $T(n) = 3T(n/2) + n \rightarrow a=3, b=2 \rightarrow n^{(\log_2 3)} \approx n^{1.585}$; $f(n) = n = O(n^{1.585-\epsilon}) \rightarrow$ **Case 1** $\rightarrow \Theta(n^{(\log_2 3)})$.

2.3 10 GATE-style PYQs

Q1 $T(n) = 2T(n/2) + n^2$ — what is $T(n)$?

ANSWER $a=2, b=2 \rightarrow n^1$; $f(n) = n^2 = \Omega(n^{1+1})$, regularity $2(n/2)^2 = n^2/2 \leq c n^2$ ✓ $\rightarrow$ Case 3 $\rightarrow \Theta(n^2)$.

Q2 $T(n) = 3T(n/3) + \sqrt{n}$ — what is $T(n)$?

ANSWER $a=3, b=3 \rightarrow n^1$; $f(n) = \sqrt{n} = O(n^{1-\epsilon}) \rightarrow$ Case 1 $\rightarrow \Theta(n)$.

Q3 $T(n) = 4T(n/2) + n^2$ — what is $T(n)$?

ANSWER $a=4, b=2 \rightarrow n^2$; $f(n) = n^2 = \Theta(n^2)$ $\rightarrow$ Case 2 $\rightarrow \Theta(n^2 \log n)$.

Q4 $T(n) = T(n/2) + 2^n$ — what is $T(n)$?

ANSWER $a=1, b=2 \rightarrow n^0 = 1$; $f(n) = 2^n$ grows faster than any polynomial, and regularity $1 \cdot 2^{n/2} \leq c \cdot 2^n$ holds for large n with $c=2^{-n/2} < 1$ $\rightarrow$ Case 3 $\rightarrow \Theta(2^n)$.

Q5 $T(n) = 16T(n/4) + n$ — what is $T(n)$?

ANSWER $a=16, b=4 \rightarrow n^2$; $f(n) = n = O(n^{2-\epsilon}) \rightarrow$ Case 1 $\rightarrow \Theta(n^2)$.

Q6 $T(n) = 2T(n/2) + n/\log n$ — what is $T(n)$?

ANSWER $f(n) = n/\log n$ is NOT polynomially smaller than n^1 (only by a log factor) — this falls in the "gap" between Case 1 and Case 2, so the basic Master theorem does NOT apply directly. (The extended/Akra–Bazzi style result gives $\Theta(n \log \log n)$.)

Q7 $T(n) = 7T(n/2) + n^2$ — what is $T(n)$? (This is the Strassen matrix-multiplication recurrence.)

ANSWER $a=7, b=2 \rightarrow n^{(\log_2 7)} \approx n^{2.807}$; $f(n) = n^2 = O(n^{2.807-\epsilon}) \rightarrow$ Case 1 $\rightarrow \Theta(n^{(\log_2 7)})$.

Q8 $T(n) = T(n-1) + n$ — what is $T(n)$?

ANSWER This is NOT in divide-and-conquer form $a \cdot T(n/b) + f(n)$ (it decreases by a constant, not divides) — Master theorem does not apply. Solve by substitution/unrolling instead: $\Theta(n^2)$.

Q9 $T(n) = 2T(n/2) + n \log n$ — what is $T(n)$?

ANSWER $a=2, b=2 \rightarrow n^1$; $f(n)=n \log n$ is NOT polynomially larger than n^1 (only by a log factor) — gap case between Case 2 and Case 3, basic Master theorem doesn't strictly apply. (Known result via recursion tree: $\Theta(n \log^2 n)$.)

Q10 $T(n) = 5T(n/5) + n$ — what is $T(n)$?

ANSWER $a=5, b=5 \rightarrow n^1$; $f(n)=n=\Theta(n^1) \rightarrow$ Case 2 $\rightarrow \Theta(n \log n)$.

2.4 10 New Practice Questions

P1 $T(n) = 9T(n/3) + n$ — Find $T(n)$.

ANSWER $a=9, b=3 \rightarrow n^2$; $f(n)=n=O(n^{2-\epsilon}) \rightarrow$ Case 1 $\rightarrow \Theta(n^2)$.

P2 $T(n) = T(n/2) + 1$ — Find $T(n)$.

ANSWER $a=1, b=2 \rightarrow n^0=1$; $f(n)=1=\Theta(n^0) \rightarrow$ Case 2 $\rightarrow \Theta(\log n)$.

P3 $T(n) = 3T(n/4) + n \log n$ — Find $T(n)$.

ANSWER $a=3, b=4 \rightarrow n^{\{\log_4 3\} \approx 0.792}$; $f(n)=n \log n = \Omega(n^{\{0.792+\epsilon\}})$; regularity $3(n/4)\log(n/4) \approx 0.75 \cdot n \log n \leq c \cdot n \log n$ for $c=0.75 < 1$ ✓ $\rightarrow$ Case 3 $\rightarrow \Theta(n \log n)$.

P4 $T(n) = 2T(n/2) + n^3$ — Find $T(n)$.

ANSWER $a=2, b=2 \rightarrow n^1$; $f(n)=n^3$ dominates heavily, regularity holds trivially $\rightarrow$ Case 3 $\rightarrow \Theta(n^3)$.

P5 $T(n) = 4T(n/2) + n \log n$ — Find $T(n)$.

ANSWER $a=4, b=2 \rightarrow n^2$; $f(n)=n \log n = O(n^{2-\epsilon})$ (polynomially smaller than n^2) $\rightarrow$ Case 1 $\rightarrow \Theta(n^2)$.

P6 $T(n) = T(n/3) + T(2n/3) + n$ — Find $T(n)$.

ANSWER Unequal-sized subproblems — Master theorem does not apply directly. Using a recursion tree: every level sums to $\Theta(n)$ of work, and the tree height is $\Theta(\log n) \rightarrow \Theta(n \log n)$.

P7 $T(n) = 27T(n/3) + n^3$ — Find $T(n)$.

ANSWER $a=27, b=3 \rightarrow n^3$; $f(n)=n^3=\Theta(n^3) \rightarrow$ Case 2 $\rightarrow \Theta(n^3 \log n)$.

P8 $T(n) = 2T(n/2) + n/2$ — Find $T(n)$.

ANSWER $f(n)=n/2=\Theta(n^1)=n^{\log_2 2} \rightarrow$ Case 2 $\rightarrow \Theta(n \log n)$ (constant factors don't change the Θ -class).

P9 $T(n) = T(2n/3) + 1$ — Find $T(n)$.

ANSWER $a=1, b=1.5 \rightarrow n^0=1$; $f(n)=1=\Theta(n^0) \rightarrow$ Case 2 $\rightarrow \Theta(\log n)$.

P10 $T(n) = 100T(n/2) + n^2$ — Find $T(n)$.

ANSWER $a=100, b=2 \rightarrow n^{(\log_2 100) \approx 6.64}$; $f(n)=n^2=O(n^{\{6.64-\epsilon\}}) \rightarrow$ Case 1 $\rightarrow \Theta(n^{(\log_2 100)}) \approx \Theta(n^{6.64})$.

3.1 Mechanics & properties (recap)

- **Data structure:** Priority queue ordered by cumulative path cost $g(n)$ (lowest cost first).
- **Strategy:** Always expand the frontier node with the smallest cumulative cost so far — a direct generalization of BFS to weighted graphs.
- **Completeness:** Yes, provided every step cost is $\geq$ a small positive constant $\epsilon > 0$ (prevents infinite sequences of ever-smaller-cost steps).
- **Optimality:** Yes — guaranteed to find the minimum-cost path, as long as costs are non-negative.
- **Time/Space complexity:** $O(b^{(1+\lceil C^*/\epsilon \rceil)})$, where C^* = cost of the optimal solution, ϵ = smallest edge cost — this can be far worse than BFS's $O(b^d)$ when edge costs are small relative to C^* .
- **UCS $\equiv$ Dijkstra's algorithm**, run until the goal node is popped from the priority queue (not merely when it's first generated/inserted — this distinction matters for correctness).
- If a cheaper path to an already-frontier node is found, UCS **updates (decrease-key)** that node's cost rather than ignoring it.

3.2 10 GATE-style PYQs

Q1 What data structure does UCS use to select the next node to expand?

(a) FIFO Queue

(b) LIFO Stack

(c) Priority Queue ordered by $g(n)$ ✓

(d) Priority Queue ordered by $h(n)$

ANSWER (c).

Q2 UCS behaves identically to BFS under which condition?

(a) When the graph is a tree

(b) When all edge costs are equal ✓

(c) When there is only one goal node

(d) Never — they are always different

ANSWER (b).

Q3 For UCS to be guaranteed complete, edge costs must satisfy:

(a) All costs equal 1

(b) Costs $\geq$ some $\epsilon > 0$ ✓

(c) Costs can be any real number, including negative

(d) Costs must be integers

ANSWER (b).

Q4 The worst-case time complexity of UCS is:

(a) $O(b^d)$

(b) $O(b^m)$

(c) $O(b^{(1+\lceil C^*/\epsilon \rceil)})$ ✓

(d) $O(bm)$

ANSWER (c).

Q5 If UCS finds a cheaper path to a node already present in the frontier, it should:

(a) Ignore the new, cheaper path

(b) Discard the node entirely

(c) Update the node's cost and reorder the priority queue ✓

(d) Restart the entire search

ANSWER (c).

Q6 UCS is most closely related to which classical graph algorithm?

(a) Kruskal's algorithm

(b) Dijkstra's algorithm ✓

(c) Bellman-Ford algorithm

(d) Floyd-Warshall algorithm

ANSWER (b) — UCS is essentially Dijkstra's algorithm run until the goal is popped.

Q7 Is UCS guaranteed to find the optimal solution if some edge costs are negative?

(a) Yes, always

(b) No — negative costs can break optimality guarantees ✓

(c) Only if there are no cycles

(d) Only if $\epsilon=0$

ANSWER (b).

Q8 At what exact point does UCS confirm a node's path cost is optimal (final)?

(a) The moment the node is generated/inserted into the frontier

(b) The moment the node is popped/removed from the priority queue for expansion ✓

(c) At the start of the search

(d) It is never truly confirmed

ANSWER (b) — this is a key subtlety separating "generated" from "expanded".

Q9 Compared to BFS on the same graph with varying (non-uniform) edge weights, UCS's node expansions can be:

(a) Always fewer than BFS

(b) Always exactly equal to BFS

(c) Potentially more, if many low-cost edges exist before reaching the goal's true cost C^* ✓

(d) Irrelevant — UCS never expands more nodes

ANSWER (c).

Q10 UCS requires a heuristic function $h(n)$ to guide its search. (a) True — like A* (b) False — UCS is an uninformed search; it uses no heuristic, only path cost $g(n)$

ANSWER (b).

3.3 10 New Practice Questions

P1 True/False: UCS always expands nodes in strictly non-decreasing order of cumulative path cost $g(n)$.

ANSWER True.

P2 If every edge in a graph has weight exactly 1, which uninformed search strategy does UCS become equivalent to?

ANSWER BFS.

P3 Write the Big-O time complexity formula for UCS in terms of branching factor b , optimal cost C^* , and minimum edge cost ϵ .

ANSWER $O(b^{(1+\lceil C^*/\epsilon \rceil)})$.

P4 Why must edge costs be bounded below by some $\epsilon > 0$ for UCS to remain complete?

ANSWER Without a positive lower bound, UCS could pursue an infinite sequence of ever-smaller-cost steps (e.g., a zero-cost cycle) and never terminate.

P5 Name one key implementation feature UCS shares with Dijkstra's algorithm.

ANSWER Both use a priority queue keyed on cumulative cost and perform a "relax/decrease-key" update when a cheaper path to a node is discovered.

P6 Is UCS guaranteed optimal when negative edge weights are present in the graph?

ANSWER No.

P7 At what moment is a node's optimal cost in UCS considered finalized?

ANSWER When the node is popped (dequeued) from the priority queue for expansion, not when it is first inserted.

P8 True/False: UCS needs a heuristic function to determine which node to expand next.

ANSWER False — UCS is uninformed and relies solely on $g(n)$, the cumulative path cost.

P9 Under what circumstance could UCS expand strictly more nodes than plain BFS would, even on the same underlying graph?

ANSWER When edge costs are small and non-uniform relative to the optimal cost C (small ϵ), forcing UCS to explore many nodes to accumulate cost before reaching C , even if BFS (counting edges, not cost) would reach a goal in fewer hops.

P10 What replaces BFS's simple FIFO queue in a UCS implementation?

ANSWER A priority queue (typically a min-heap) ordered by cumulative path cost $g(n)$.

Quick Revision Checklist

- ☐ Can classify any nested-loop snippet in under 30 seconds using the multiply/add/log rules
- ☐ Know the harmonic-series trap ($\Theta(n \log n)$) and the geometric-shrinking trap ($\Theta(n)$, not $n \log n$)
- ☐ Can apply all 3 Master theorem cases confidently, including checking the regularity condition for Case 3
- ☐ Can recognize when Master theorem does NOT apply (unequal subproblems, non divide-and-conquer form, polynomial-gap failures)
- ☐ Know UCS's data structure, completeness/optimalty conditions, and its exact equivalence to Dijkstra's algorithm
- ☐ Understand the "generated vs. expanded" subtlety for confirming optimal cost in UCS